UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE COMPUTAÇÃO E INFORMÁTICA

Filipe Soares – RA 10734981
Miguel Soares – RA 10734935
Julio Freire – RA 10734961

PROJETO EXTENSIONISTA: SISTEMA INTEGRADO PARA A INDÚSTRIA FARMACÊUTICA

Documento 3 – Modelagem UML
Diagramas UML do recorte funcional e respectivas descrições

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO.....3

2 DIAGRAMA DE CLASSES ADAPTADO AO PADRÃO MVC.....3

2.1 Decisões de modelagem e distribuição nas camadas MVC.....3

2.2 Especificação das classes por camada.....6

2.3 Relacionamentos entre as classes.....7

2.4 Enumerações e valores utilizados.....7

2.5 Especificação PlantUML do diagrama de classes.....7

3 CASOS DE USO DO RECORTE.....9

4 ESTADOS DO LOTE DE MATERIAL.....10

REFERÊNCIAS.....12

1 INTRODUÇÃO

Este documento reúne a modelagem do Sistema Integrado para a Indústria Farmacêutica (RastroFarma) na Unified Modeling Language — UML (OBJECT MANAGEMENT GROUP, 2017). A modelagem refere-se ao recorte funcional de controle de qualidade sobre lotes de matéria-prima recebidos em quarentena, correspondente ao requisito funcional RF-QUA-01 e ao caso de uso UC03 (Avaliar lote de material).

O recorte é representado por três diagramas: o Diagrama de Classes adaptado ao padrão MVC (Figura 1), especificado formalmente em PlantUML; o Diagrama de Casos de Uso (Figura 2), com os atores Almoxarife e Analista da Qualidade; e o Diagrama de Estados do lote de material (Figura 3). Conforme as convenções do projeto, os identificadores de classes, atributos, métodos e estados são preservados exatamente como no código, com os estados em letras maiúsculas (QUARENTENA, APROVADO, REPROVADO).

2 DIAGRAMA DE CLASSES ADAPTADO AO PADRÃO MVC

Na modelagem UML, o recorte de avaliação de qualidade articula as classes da vista de materiais, qualidade e estoque com a classe de controle de acesso. Sob a arquitetura MVC, essas classes são distribuídas nas três camadas estruturais, garantindo que as regras de estado permaneçam encapsuladas no Model.

2.1 Decisões de modelagem e distribuição nas camadas MVC

A adaptação do diagrama de classes para o padrão MVC segue quatro decisões estruturais diretamente alinhadas à especificação original:

a) **encapsulamento no Model:** a classe LoteMaterial concentra o atributo status (do tipo enumerado StatusLoteMaterial) e os métodos de negócio aprovar(), reprovar(), estaVencido(), podeSerConsumido() e baixarSaldo(). O lote nasce em QUARENTENA e somente transita para APROVADO ou REPROVADO mediante validação de estado e de permissão;

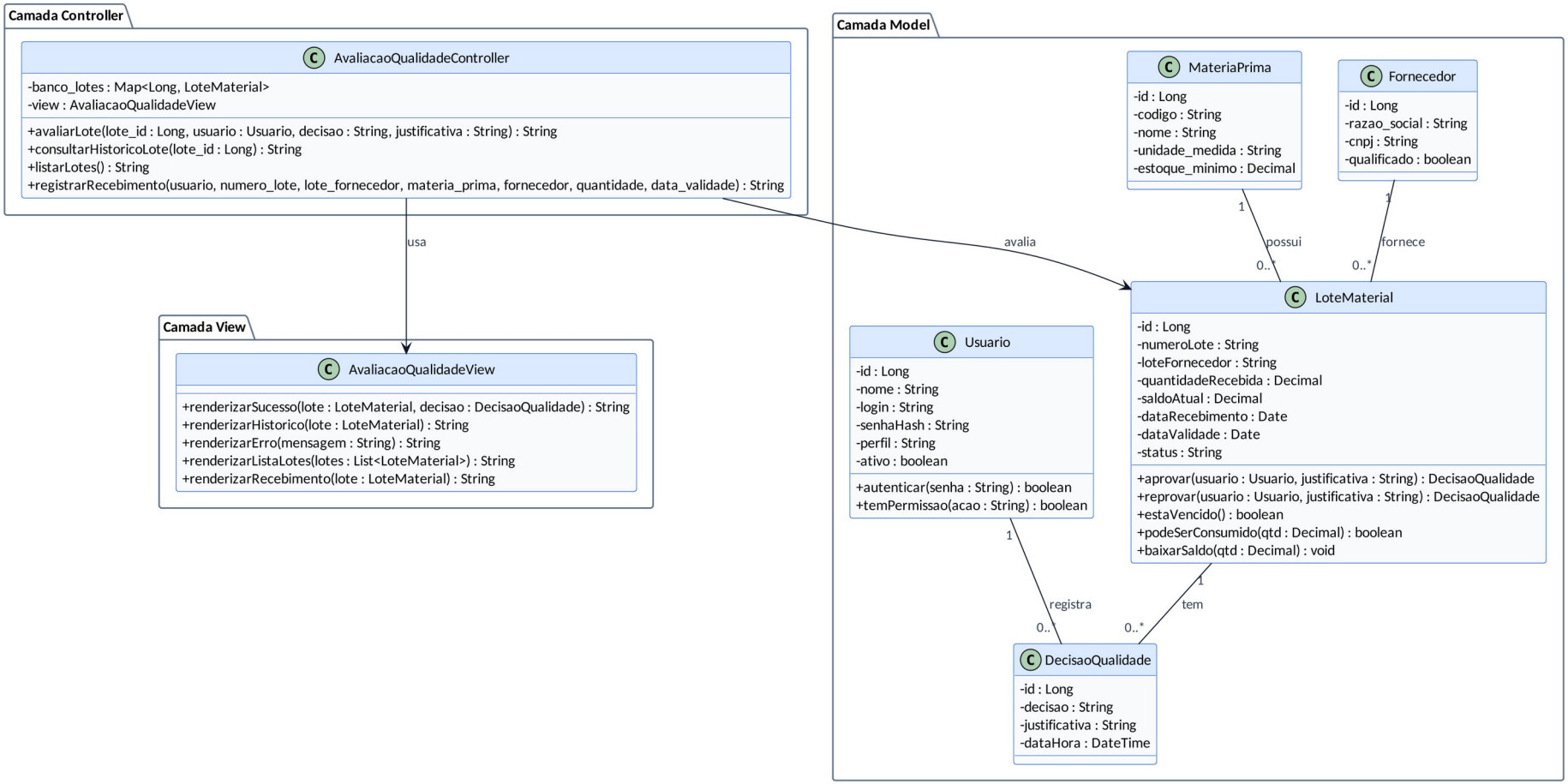
b) **rastreabilidade de decisões e perfis no Model:** cada avaliação gera uma instância de DecisaoQualidade vinculada ao LoteMaterial e ao Usuario que a registrou, armazenando a enumeração Decisao (APROVADO ou REPROVADO), a justificativa e o carimbo de data e hora;

c) **isolamento da apresentação na View:** a classe AvaliacaoQualidadeView não altera estados nem acessa o banco de dados, limitando-se a receber os objetos de domínio e formatar respostas estruturadas em JSON para exibição no console;

d) **orquestração no Controller:** a classe AvaliacaoQualidadeController expõe os métodos acionados pelo usuário (avaliarLoteMaterial e consultarHistoricoLote), recupera as entidades de domínio, invoca os métodos do Model e repassa o resultado para a View.

A Figura 1, na página seguinte, apresenta o Diagrama de Classes do recorte, gerado a partir da especificação PlantUML da seção 2.5.

Figura 1 – Diagrama de Classes do recorte funcional adaptado ao padrão MVC



Fonte: Elaborado pelos autores (gerado a partir da especificação PlantUML).

2.2 Especificação das classes por camada

O Quadro 1 detalha as classes e enumerações do recorte, com seus atributos principais e operações, organizadas por camada MVC.

Quadro 1 – Especificação das classes do recorte funcional por camada MVC

Camada MVC	Classe / Enumeração	Atributos principais	Operações (métodos)
Model	LoteMaterial	id: Long, numeroLote: String, loteFornecedor: String, quantidadeRecebida: Decimal, saldoAtual: Decimal, dataRecebimento: Date, dataValidade: Date, status: StatusLoteMaterial	+aprovar(): void +reprovar(): void +estaVencido(): boolean +podeSerConsumido(qtd: Decimal): boolean +baixarSaldo(qtd: Decimal): void
Model	DecisaoQualidade	id: Long, decisao: Decisao, justificativa: String, dataHora: DateTime	Construtor de registro imutável vinculado a Usuario e LoteMaterial.
Model	MateriaPrima	id: Long, codigo: String, nome: String, unidadeMedida: String, estoqueMinimo: Decimal	Entidade cadastral associada a LoteMaterial (1 possui 0..*).
Model	Fornecedor	id: Long, razaoSocial: String, cnpj: String, qualificado: boolean	Entidade cadastral associada a LoteMaterial (1 fornece 0..*).
Model	Usuario	id: Long, nome: String, login: String, senhaHash: String, perfil: PerfilUsuario, ativo: boolean	+autenticar(senha: String): boolean +temPermissao(acao: String): boolean
Model	StatusLoteMaterial, Decisao, PerfilUsuario	Valores enumerados restritos às regras de negócio do domínio farmacêutico.	Restrição estática de estados e perfis.
View	AvaliacaoQualidadeView	Sem estado de negócio.	+renderizarSucesso(lote, decisao): String +renderizarHistorico(lote): String +renderizarErro(mensagem): String +renderizarListaLotes(lotes): String +renderizarRecebimento(lote): String
Controller	AvaliacaoQualidadeController	banco_lotes: Map<Long, LoteMaterial>, view: AvaliacaoQualidadeView	+avaliarLote(lote_id, usuario, decisao, justificativa): String +consultarHistoricoLote(lote_id): String +listarLotes(): String +registrarRecebimento(usuario, numero_lote, lote_fornecedor, materia_prima, fornecedor, quantidade, data_validade): String

Fonte: Elaborado pelos autores.

2.3 Relacionamentos entre as classes

O Quadro 2 descreve os relacionamentos representados no diagrama, com suas multiplicidades.

Quadro 2 – Relacionamentos do Diagrama de Classes

Classe de origem	Relacionamento	Classe de destino	Descrição
Fornecedor (1)	Associação "fornece"	LoteMaterial (0..*)	Um fornecedor fornece zero ou mais lotes de material.
MateriaPrima (1)	Associação "possui"	LoteMaterial (0..*)	Uma matéria-prima possui zero ou mais lotes de material.
LoteMaterial (1)	Associação "tem"	DecisaoQualidade (0..*)	Um lote tem zero ou mais decisões de qualidade (histórico).
Usuario (1)	Associação "registra"	DecisaoQualidade (0..*)	Um usuário registra zero ou mais decisões de qualidade.
AvaliacaoQualidadeController	Associação direcionada "usa"	AvaliacaoQualidadeView	O Controller usa a View para renderizar as respostas.
AvaliacaoQualidadeController	Associação direcionada "avalia"	LoteMaterial	O Controller aciona a avaliação do lote no Model.

Fonte: Elaborado pelos autores.

2.4 Enumerações e valores utilizados

O Quadro 3 relaciona os valores das enumerações empregados no código-fonte e nos cenários de teste do recorte.

Quadro 3 – Enumerações do recorte e valores utilizados no código

Enumeração	Valores	Uso
StatusLoteMaterial	QUARENTENA, APROVADO, REPROVADO, VENCIDO, ESGOTADO	Status do LoteMaterial ao longo do ciclo de avaliação e consumo.
Decisao	APROVADO, REPROVADO	Resultado registrado em DecisaoQualidade.
PerfilUsuario	QUALIDADE, ALMOXARIFE	Perfil do Usuario, usado no controle de acesso (temPermissao): QUALIDADE avalia lotes; ALMOXARIFE registra o recebimento.

Fonte: Elaborado pelos autores.

2.5 Especificação PlantUML do diagrama de classes

Em conformidade com o padrão de documentação adotado no projeto, apresenta-se a seguir a descrição formal em PlantUML do Diagrama de Classes do recorte funcional estruturado nas camadas Model, View e Controller.

Listagem 1 – Especificação PlantUML do Diagrama de Classes MVC

```
@startuml
skinparam classAttributeIconSize 0

package "Camada Controller" {
    class AvaliacaoQualidadeController {
        - banco_lotes : Map<Long, LoteMaterial>
        - view : AvaliacaoQualidadeView
        + avaliarLote(lote_id : Long, usuario : Usuario, decisao : String, justificativa :
String) : String
        + consultarHistoricoLote(lote_id : Long) : String
        + listarLotes() : String
        + registrarRecebimento(usuario, numero_lote, lote_fornecedor, materia_prima,
fornecedor, quantidade, data_validade) : String
    }
}

package "Camada View" {
    class AvaliacaoQualidadeView {
        + renderizarSucesso(lote : LoteMaterial, decisao : DecisaoQualidade) : String
        + renderizarHistorico(lote : LoteMaterial) : String
        + renderizarErro(mensagem : String) : String
        + renderizarListaLotes(lotes : List<LoteMaterial>) : String
        + renderizarRecebimento(lote : LoteMaterial) : String
    }
}

package "Camada Model" {
    class Fornecedor {
        - id : Long
        - razao_social : String
        - cnpj : String
        - qualificado : boolean
    }

    class MateriaPrima {
        - id : Long
        - codigo : String
        - nome : String
        - unidade_medida : String
        - estoque_minimo : Decimal
    }

    class LoteMaterial {
        - id : Long
        - numeroLote : String
        - loteFornecedor : String
        - quantidadeRecebida : Decimal
        - saldoAtual : Decimal
        - dataRecebimento : Date
        - dataValidade : Date
        - status : String
        + aprovar(usuario : Usuario, justificativa : String) : DecisaoQualidade
        + reprovar(usuario : Usuario, justificativa : String) : DecisaoQualidade
        + estaVencido() : boolean
        + podeSerConsumido(qtd : Decimal) : boolean
        + baixarSaldo(qtd : Decimal) : void
```

```
    }

    class DecisaoQualidade {
        - id : Long
        - decisao : String
        - justificativa : String
        - dataHora : DateTime
    }

    class Usuario {
        - id : Long
        - nome : String
        - login : String
        - senhaHash : String
        - perfil : String
        - ativo : boolean
        + autenticar(senha : String) : boolean
        + temPermissao(acao : String) : boolean
    }
}

Fornecedor "1" -- "0..*" LoteMaterial : fornece
MateriaPrima "1" -- "0..*" LoteMaterial : possui
LoteMaterial "1" -- "0..*" DecisaoQualidade : tem
Usuario "1" -- "0..*" DecisaoQualidade : registra
AvaliacaoQualidadeController --> AvaliacaoQualidadeView : usa
AvaliacaoQualidadeController --> LoteMaterial : avalia

@enduml
```

Fonte: Elaborado pelos autores.

3 CASOS DE USO DO RECORTE

O recorte funcional envolve dois atores e quatro casos de uso. O Almoxarife registra o recebimento do lote, que entra em QUARENTENA; o Analista da Qualidade avalia o lote (UC03); e ambos podem consultar o histórico de decisões. O caso de uso UC14 estende (*extend*) o caso de uso base UC03: quando a decisão da qualidade em UC03 for negativa, UC14 registra a reprovação com justificativa obrigatória.

Quadro 4 – Casos de uso do recorte funcional

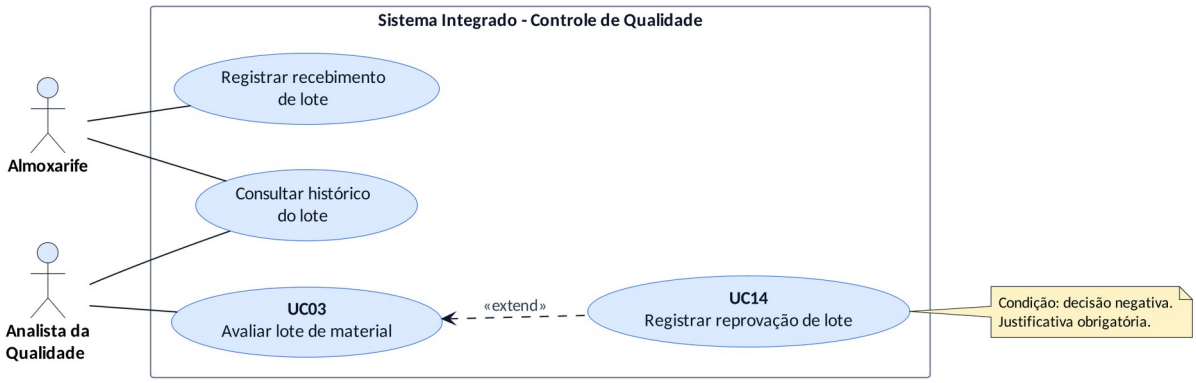
Código	Nome	Tipo	Ator principal	Descrição
—	Registrar recebimento de lote	Caso de uso	Almoxarife	Registra um novo lote de matéria-prima recebido do fornecedor; o lote entra com status QUARENTENA, bloqueado para consumo. Restrito ao perfil ALMOXARIFE.
UC03	Avaliar lote de material	Caso de uso base	Analista da Qualidade	Aprova ou reprova o lote em quarentena; na aprovação, o status muda para APROVADO.

Código	Nome	Tipo	Ator principal	Descrição
UC14	Registrar reprovação de lote	Caso de uso de extensão (<<extend>> de UC03)	Analista da Qualidade	Quando o lote é reprovado, exige justificativa e muda o status para REPROVADO, bloqueando o consumo.
—	Consultar histórico do lote	Caso de uso	Analista da Qualidade e Almoxarife	Exibe as decisões de qualidade do lote, com usuário, data e hora, decisão e justificativa (RF-QUA-03).

Fonte: Elaborado pelos autores.

A Figura 2 apresenta o Diagrama de Casos de Uso do recorte. Os atores, representados pelos bonecos, ligam-se aos casos de uso que executam, e o UC14 se liga ao UC03 pela relação <<extend>>, cuja condição aparece na nota.

Figura 2 – Diagrama de Casos de Uso do recorte funcional



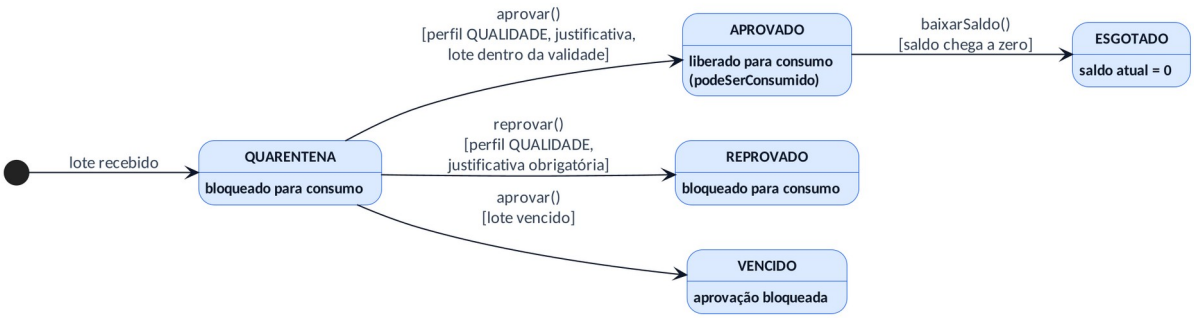
Fonte: Elaborado pelos autores.

Os casos de uso UC03, UC14 e Consultar histórico do lote estão relacionados aos requisitos RF-QUA-01 (aprovar ou reprovar lote em QUARENTENA, registrando a justificativa), RF-QUA-03 (histórico das decisões de qualidade), RNF-03 (ação restrita ao perfil QUALIDADE) e RNF-06 (rejeição de entradas inválidas ou campos obrigatórios vazios), especificados no Documento 1 – Engenharia de Requisitos.

4 ESTADOS DO LOTE DE MATERIAL

A máquina de estados do lote de material, descrita na apresentação do projeto junto aos casos de uso UC03 e UC14, é implementada nos métodos da classe LoteMaterial. A Figura 3 apresenta o Diagrama de Estados correspondente, e o Quadro 5 detalha cada transição.

Figura 3 – Diagrama de Estados do LoteMaterial



Fonte: Elaborado pelos autores.

Quadro 5 – Transições de estado do LoteMaterial

Estado de origem	Evento / condição	Estado de destino
(recebimento)	registrarRecebimento(): Almoxarife registra o lote recebido do fornecedor.	QUARENTENA
QUARENTENA	aprovar(): usuário com perfil QUALIDADE, justificativa informada e lote dentro da validade (UC03).	APROVADO
QUARENTENA	reprovar(): usuário com perfil QUALIDADE e justificativa obrigatória informada (UC03 / UC14).	REPROVADO
QUARENTENA	aprovar() sobre lote com validade expirada (estaVencido()); a aprovação é bloqueada.	VENCIDO
APROVADO	baixarSaldo(): consumo que leva o saldo atual a zero.	ESGOTADO

Fonte: Elaborado pelos autores.

Lotes fora do estado QUARENTENA não podem ser avaliados novamente, e somente lotes APROVADOS, dentro da validade e com saldo suficiente podem ser consumidos (podeSerConsumido()).

REFERÊNCIAS

OBJECT MANAGEMENT GROUP. **Unified Modeling Language (UML)**: specification, version 2.5.1. Needham: OMG, 2017.